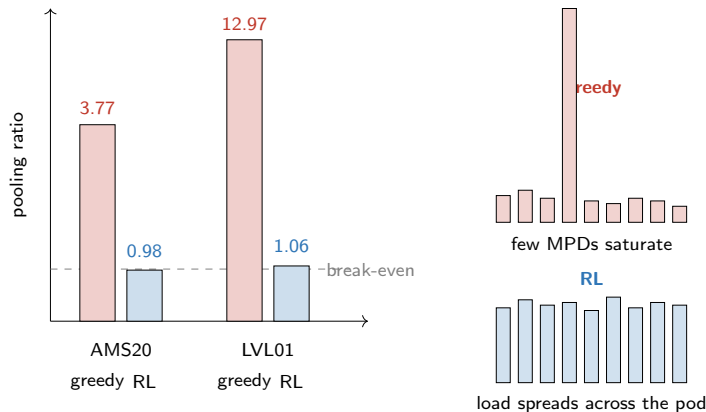


Reward Shaping and Training Throughput

Pulak Mehrotra

April 2026

Recap: RL beat greedy by spreading load much more effectively.



Held-out traces, 50 random pod mappings each.

RL is the first policy near break-even on both held-out traces, and it gets there by avoiding the hotspot pattern that greedy creates.

Recap: the remaining problem is reward shaping, not just more training.

- ▶ The old reward watched a narrow proxy, so the agent could improve the score without improving final pooling savings.
- ▶ The newer projected-load reward fixed the worst collapse mode and made training stable.
- ▶ But savings still plateau around **1–2%** early and then stop moving.
- ▶ So the next bottleneck is **reward alignment**: what signal should the agent actually optimize?

Interpretation

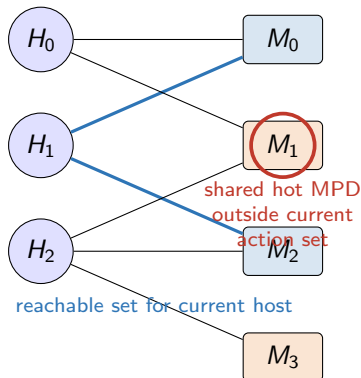
The current reward is a *better proxy*, not the true objective.

Better proxy $\neq$ optimal control signal.

What that means

More timesteps alone are unlikely to fix the plateau.

New reward question 1: how much should reachability constrain the reward?



Current decision

Host H_1 can only act on $N(H_1) = \{M_0, M_2\}$.

Shared bottleneck

M_1 can still become the hot MPD because it is shared by other hosts.

Reward choice

Score only the local neighborhood, or add pod-wide context?

New reward question 2: proxy reward vs. actual goal.

Proxy rewards score each step	Actual objective scores the whole episode
post-placement peak projected load after near-term departures local or global congestion proxy	$\text{pooling_savings} = 1 - \frac{m \cdot \max_j c_j^{\text{peak}}}{D_{\text{pod}}}$
Why they help	Why it matters
dense signal easier credit assignment faster learning	a step can look locally good and still create the final episode peak

Tension: proxy rewards are learnable, but terminal pooling savings is the quantity we actually care about.

New reward question 3: can the optimal solver provide a better shaping signal?

- ▶ The codebase already has an exact snapshot solver in `octopus/optimal.py`.
- ▶ For a given demand vector, it gives the **lowest achievable peak** under the topology constraints.
- ▶ That creates an **optimality-gap** signal:

$$\delta_t = \frac{c_{\max}^{\text{post}} - t^*(s_t)}{D_{\text{pod}}}$$

- ▶ If useful, this is a cleaner shaping term than hand-designed heuristics because it is anchored to a real lower bound.

Why this is attractive

It tells the policy not just “did you avoid a hot MPD?” but “how far are you from the best possible placement right now?”

Why this is risky

It requires a max-flow solve. That makes it the most informative reward candidate and the most expensive one.

Putting the reward options together.

R1: score only the worst MPD after each placement



R2: score all MPDs the current host can reach, with departures



R3: add a pod-wide congestion term beyond the local neighborhood



R4: ignore step rewards and score only final pooling savings

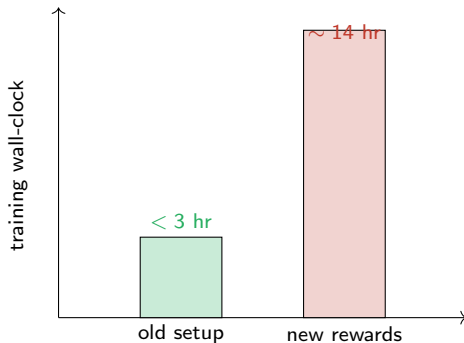


R4 + PBRS: keep the true objective, but add dense shaping to help learning



Optimal gap: compare each snapshot to the exact best achievable placement

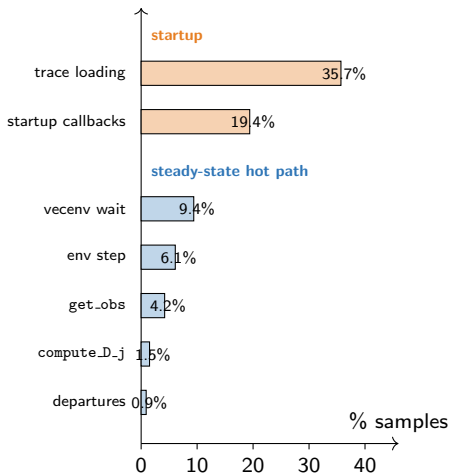
New rewards turned training into a systems problem.



Takeaway

- ▶ Rewards became more informative, but experiment turnaround became much slower.
- ▶ The main blocker is throughput: each reward iteration now carries substantial wall-clock cost.

Quick profile: where time goes and what we can remove.



py-spy, main process, short probe (2k steps).

Startup → cache it

Trace loading + callbacks dominate short ablation runs.

Fix: **multi-trace precompute cache**, reused across `reset()`.

Hot path → vectorize it

`get_obs`, `compute_D_j`,
`departures` all walk
`mpd_vm_allocs` in Python.
Fix: **flat-array MPD state**,
NumPy hot kernels.

Proposed plan: attack the two bottlenecks directly.

